

Teaching Exam

Subject – Computer Science

Topic – Digital Logic

Lecture No.- 11



By- Rahul Sir

Topics to be Covered

Topic



- 1:- K-Map using Don't CARE ✓
- 2:- Introduction to Combinational Circuit ✓
- 3:- Types of Combinational Circuit ✓
- 4:- Binary Adder ✓
- 5:- Half Adder and its Application ✓
- 6:- Full Adder and its Application ✓
- 7:- Binary Subtractor ✓
- 8:- Half Subtractor ✓



The Concept of “Don’t Cares”

- The six implicants on the K-Map shown can be represented by the simplified expression $f = x y + x z$.
- Suppose for our particular logic system, we know that y and z will never be 0 together.
- Then the yz implicants do not matter, since they cannot happen.
- To show the condition cannot occur, we put X 's in the column -- they are “don't cares” -- conditions that cannot happen.

	$\overline{y}z$	$\overline{y}z$	yz	yz
$\overline{W}X$	0000 0 X 0	0001 1 1	0011 3 3	0010 2 2
$\overline{W}X$	0100 4 X 4	0101 5 1 5	0111 7 1 7	0110 6 1 6
WX	1100 C X 12	1101 D 1 13	1111 F 1 15	1110 E 1 14
WX	1000 8 X 8	1001 9 9	1011 B 11	1010 A 10

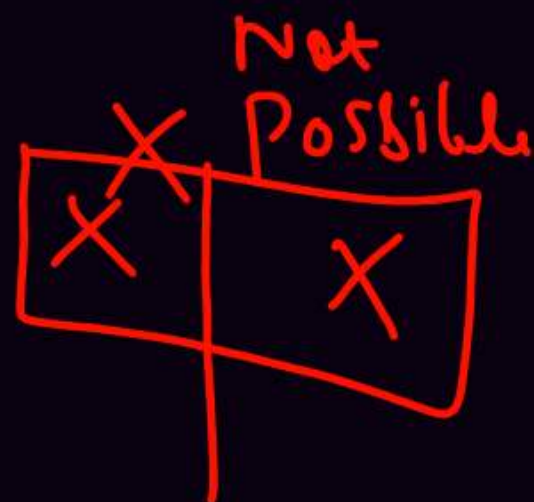


Don't Care (X)

$$Y = \sum(m_1, m_7, m_5, m_3) + d(4, 2)$$

X

SOP
X=1



A \ BC	00	01	11	10
0	0	1	1	X
1	X	1	1	

0 1 3 2
4 5 7 C

$$Y = C + \bar{A}B + A\bar{B}$$

$$Y = C + A \oplus B$$

$$f = \prod (m_1, m_5, m_6, m_{11}, m_{12}, m_{13}, m_{14}) + d(4)$$

$y = (\bar{B} + C) \cdot (\bar{B} + D)$
 $(A + C + \bar{D}) \cdot$
 $(\bar{A} + B + \bar{C} + \bar{D})$

AB \ CD	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

Karnaugh map showing minterms (circles) and groupings (red and blue lines). The map is labeled with AB and CD. The minterms are circled in the cells (0,1), (0,5), (1,12), (1,13), (1,14), and (1,11). The groupings are: a red rectangle covering (0,1), (0,5), (1,12), and (1,13); a blue rectangle covering (0,1), (0,5), (1,14), and (1,10); a blue circle around (1,11); and a blue circle around (1,14).



“Don’t Cares” (2)

- Since the function will never get into the left column, we “don’t care” how those minterms are represented.

Why not make them 1's?

- Doing that, we can make a much larger prime implicant and a simpler Boolean expression. **For this implicant, $f = X$.**

- The expression is valid, since the forbidden condition will never allow the two left squares to be occupied.
- “Don’t cares” let us further simplify an expression.

	$\overline{y}z$	$\overline{y}z$	yz	$y\overline{z}$
$\overline{WX}$	0000 0 1 0	0001 1 1	0011 3 3	0010 2 2
WX	0100 4 1 4	0101 5 1 5	0111 7 1 7	0110 6 1 6
WX	1100 C 1 12	1101 D 1 12	1111 F 1 12	1110 E 1 11
WX	1000 8 1 8	1001 9 9	1011 B 11	1010 A 10



“Don’t Cares” – Another Example

- Assume the Boolean expression is as shown on the K-map (black 1's and black prime implicants).
- The simplest SOP expressions for the function is $f = wxz + wxy$.
- Also assume that $wxyz$ and $wxy\bar{z}$ cannot occur.
- Since these cannot ever happen, they are “don’t cares,” and since they are “don’t cares,” make them 1 (**red**)!
- We can then further simplify the expression to $f = wx + xz$ (larger prime implicants).

	$\bar{y}z$	$\bar{y}z$	yz	$y\bar{z}$
$\bar{w}x$	0000 0 0	0001 1 1	0011 3 3	0010 2 2
$\bar{w}x$	0100 4 4	0101 5 5	0111 7 7	0110 6 6
wx	1100 C 12	1101 D 13	1111 F 15	1110 E 14
wx	1000 8 8	1001 9 9	1011 B 11	1010 A 10



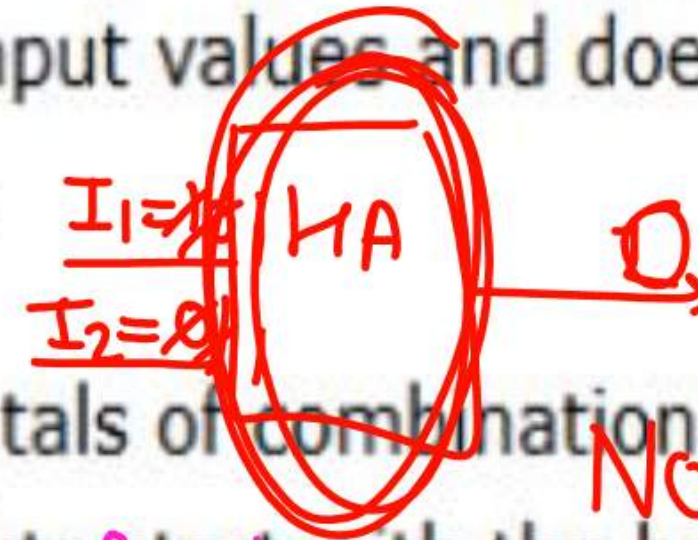
“Don’t Cares” -- Summary

- “Don’t Cares” occur when there are variable combinations, represented by squares on a Karnaugh map, that cannot occur in a digital circuit or Boolean expression.
- Such a square is called a “Don’t Care.” Since it can never happen, we can assign a value of 1 to the square and use that 1 to (possibly) construct larger prime implicants, further simplifying the circuit (you could assign it the value 0 in a POS representation).
- Circuits or Boolean expressions derived using “Don’t Care” squares are as valid as any other expression or circuit.
- We will see later in sequential logic that the concept of “Don’t Cares” will help in simplifying counter circuits.



A **combinational circuit**, also called a **combinational logic circuit**, is a digital electronic circuit whose output is determined by present inputs only.

The output of a combinational logic circuit does not depend on the history of the circuit operation. In other words, a combinational circuit is a digital logic circuit whose output depends only on the present input values and does not depend on any feedback or previous input or output values.



In this chapter, we will explain the fundamentals of combinational circuits, and its block diagram, types, and applications. So, let's start with the basic definition of combinational circuits.

No feedback



What is a Combinational Circuit?

A combinational circuit is a type of digital logic circuit whose output depends on the present input values only and does not depend on past input and output values. Therefore, a combinational circuit is considered to not have a memory element in its circuit that stores previous inputs and outputs. Instead, it consists of a certain number of input lines to apply current input values and a certain number of output lines.

The most important characteristic of a combinational circuit is that it does not have any feedback path between input and output. Therefore, the combinational circuits can be categorized as open-loop systems.



Block Diagram of Combinational Circuit

The following figure depicts the block diagram of a combinational logic circuit.



Here, we can see that there are only three key elements in the circuit diagram of a combinational circuit, they are –

- **Input Lines** – The input lines are used to enter the input values into the combinational circuit.
- **Processing Unit** – It is the main element that processes the input values depending on the type of the circuit. For example, a full adder adds three binary bits.
- **Output Lines** – The output lines are used to take results generated by the circuit.



Characteristics of Combinational Circuits

The following are the main characteristics of combinational circuits –

- The output of a combinational circuit, at any instant of time, depends only on the present input values at that instant of time.
- Combinational circuits do not use any kind of memory element in their circuits. Thus, the previous state of input and output values do not have any effect on the present operation of the circuit.
- The output of a combinational circuit can be entirely predicted using its logical operation and input values.
- Combinational circuits produce an instantaneous output in response to any change in its input values.



Types of Combinational Circuits

In digital electronics, the combinational circuits are important components of digital systems. Depending on the functions performed, there are various types of combinational circuits. Some common types of combinational circuits and their functions are explained below –

- Binary Adders ✓
- Binary Subtractors ✓
- Multiplexers (MUX) ✓
- Demultiplexers (DEMUX) ✓
- Encoders ✓
- Decoders ✓
- Comparators ✓



Addition is one of the most basic operations performed by different electronic devices like computers, calculators, etc. The electronic circuit that performs the addition of two or more numbers, more specifically binary numbers, is called as **adder**. Since, the logic circuits use binary number system to perform the operations, hence the adder is referred to as **binary adder**

Depending on the number of bits that the circuit can add, adders (or binary adders) are of two types –

- Half Adder ✓
- Full Adder ✓



What is a Half-Adder?

A combinational logic circuit which is designed to add two binary digits is called as a **half adder**. The half adder provides the output along with a carry value (if any). The half adder circuit is designed by connecting an EX-OR gate and one AND gate. It has two input terminals and two output terminals for sum and carry. The block diagram and circuit diagram of a half adder are shown in Figure-1.

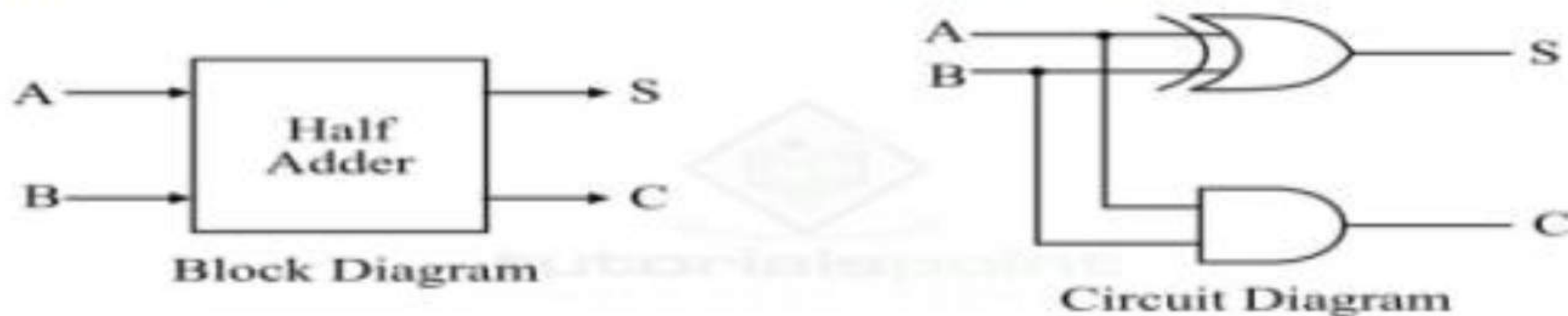


Figure 1 - Half Adder

From the logic circuit diagram of half adder, it is clear that A and B are the two input bits, S is the output sum, and C is the output carry bit.

In the case of a half adder, the output of the EX-OR gate is the sum of two bits and the output of the AND gate is the carry. Although, the carry obtained in one addition will not be forwarded in the next addition because of this it is known as half adder.

Note:- Carry Denote as MSB Bit and Sum Denote as LSB Bit



Truth Table of Half Adder

Truth table is one that gives the relationship between inputs and outputs of a logic circuit and explains the operation of the circuit. The following is the truth table of the half-adder –

Inputs		Outputs	
A	B	S (Sum)	C (Carry)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

K-Map for Half Adder

We can use the K-Map (Karnaugh Map), a method for simplifying Boolean algebra, to determine equations of the sum bit (S) and the output carry bit (C) of the half adder circuit.

The k-map for half adder circuit is shown in Figure-2.

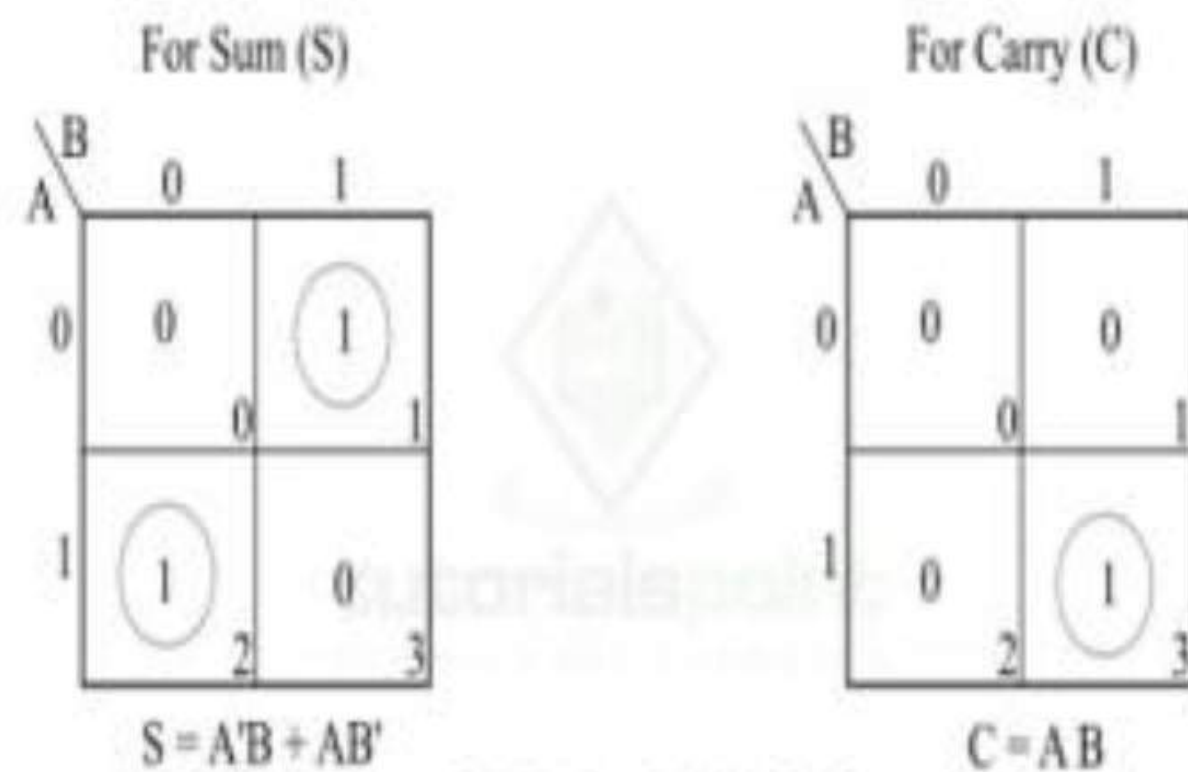


Figure 2 - K-Map for Half Adder

Note:- Carry Denote as MSB Bit and Sum Denote as LSB Bit



Characteristic Equations of Half-Adder

The characteristic equations of half adder, i.e., equations of sum (S) and carry (C) are obtained according to the rules of binary addition. These equations are given below –

The sum (S) of the half-adder is the XOR of A and B. Thus,

$$\text{Sum, } S = A \oplus B = AB' + A'B$$

The carry (C) of the half-adder is the AND of A and B. Therefore,

$$\text{Carry, } C = A \cdot B$$

Applications of Half Adder

The following are some important applications of half adder –

- Half adder is used in ALU (Arithmetic Logic Unit) of computer processors to add binary bits.
- Half adder is used to realize full adder circuit.
- Half adder is used in calculators.
- Half adder is used to calculate addresses and tables.



•Address Calculation:

In computer systems, addresses are often represented in binary format. When incrementing or decrementing an address, a series of binary additions or subtractions are performed. Half adders, as part of a larger adder circuit, are used in these calculations.

Table Indexing:

Similarly, when accessing data within a table, the index (which is a numerical value) needs to be calculated. Half adders can be part of the logic that determines the correct memory location to access within the table.

ALU Operations:

Processors use Arithmetic Logic Units (ALUs) for various operations, including address calculations. These ALUs incorporate half adders as basic components to perform binary additions, which are essential for address and table manipulation.



What is a Full Adder?

A combinational logic circuit that can add two binary digits (bits) and a carry bit, and produces a sum bit and a carry bit as output is known as a **full-adder**.

In other words, a combinational circuit which is designed to add three binary digits and produces two outputs (sum and carry) is known as a full adder. Thus, a full adder circuit adds three binary digits, where two are the inputs and one is the carry forwarded from the previous addition. The block diagram and circuit diagram of the full adder are shown in Figure-1.

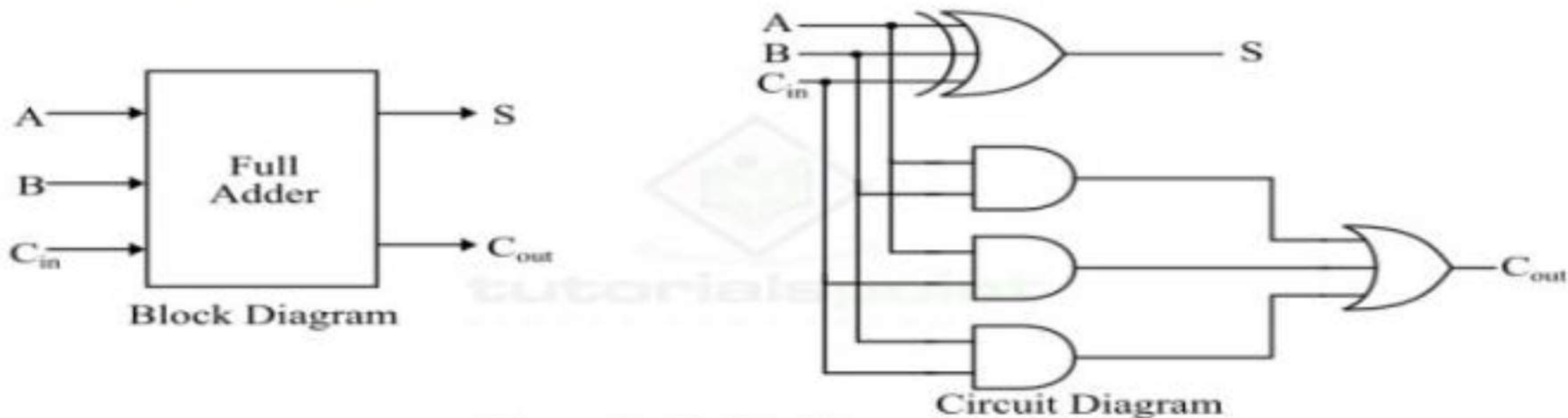


Figure 1 - Full Adder



Operation of Full Adder

Full adder takes three inputs namely A , B , and C_{in} . Where, A and B are the two binary digits, and C_{in} is the carry bit from the previous stage of binary addition. The sum output of the full adder is obtained by XORing the bits A , B , and C_{in} . While the carry output bit (C_{out}) is obtained using AND and OR operations.

Note:- Carry Denote as MSB Bit and Sum Denote as LSB Bit



Truth Table of Full Adder

Truth table is one that indicates the relationship between input and output variables of a logic circuit and explains the operation of the logic circuit. The following is the truth table of the full-adder circuit –

Inputs			Outputs	
A	B	C _{in}	S (Sum)	C _{out} (Carry)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



K-Map for Full Adder

K-Map (Karnaugh Map) is a tool for simplifying binary complex Boolean algebraic expressions. The K-Map for full adder is shown in Figure-2.

For Sum (S)

A \ BC _{in}				
	00	01	11	10
0	0	1	0	1
1	1	0	1	0

$$S = A'B'C_{in} + A'BC'_{in} + AB'C'_{in} + ABC_{in}$$

For Output Carry (C_{out})

A \ BC _{in}				
	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$C_{out} = AB + AC_{in} + BC_{in}$$



Characteristic Equations of Full Adder

The characteristic equations of the full adder, i.e. equations of sum (S) and carry output (C_{out}) are obtained according to the rules of binary addition. These equations are given below –

The sum (S) of the full-adder is the XOR of A, B, and C_{in} . Therefore,

$$\text{Sum, } S = A \oplus B \oplus C_{in} = A'B'C_{in} + A'BC'_{in} + AB'C'_{in} + ABC_{in}$$

The carry (C) of the half-adder is the AND of A and B. Therefore,

$$\text{Carry, } C = AB + AC_{in} + BC_{in}$$



Advantages of Full Adder

The following are the important advantages of full adder over half adder –

- Full adder provides facility to add the carry from the previous stage.
- The power consumed by the full adder is relatively less as compared to half adder.
- Full adder can be easily converted into a half subtractor just by adding a NOT gate in the circuit.
- Full adder produces higher output than half adder.
- Full adder is one of the essential parts of critical digital circuits like multiplexers.
- Full adder performs operation at higher speed.



Applications of Full Adder

The following are the important applications of full adder –

- Full adders are used in ALUs (arithmetic logic units) of CPUs of computers.
- Full adders are used in calculators.
- Full adders also help in carrying out multiplication of binary numbers.
- Full adders are also used to realize critical digital circuits like multiplexers.
- Full adders are used to generate memory addresses.
- Full adders are also used in generation of program counterpoints.
- Full adders are also used in GPU (Graphical Processing Unit).



In digital electronics, a **subtractor** is a combinational logic circuit that can perform the subtraction of two number (binary numbers) and produce the difference between them. It is a combinational circuit that means its output depends on its present inputs only. Although, in practice, the subtraction of two binary number is accomplished by taking the 1's or 2's compliment of the subtrahend and adding it to the minuend.

In this way, the subtraction operation of binary numbers can be converted into simple addition operation which makes hardware construction simple and less expensive. There are two types of subtractors namely, **Half Subtractor** and **Full Subtractor**.



A **half-subtractor** is a combinational logic circuit that have two inputs and two outputs (i.e. difference and borrow). The half subtractor produces the difference between the two binary bits at the input and also produces a borrow output (if any). In the subtraction ($A-B$), A is called as **Minuend bit** and B is called as **Subtrahend bit**. The block diagram and logic circuit diagram of the half subtractor is shown in Figure-1.

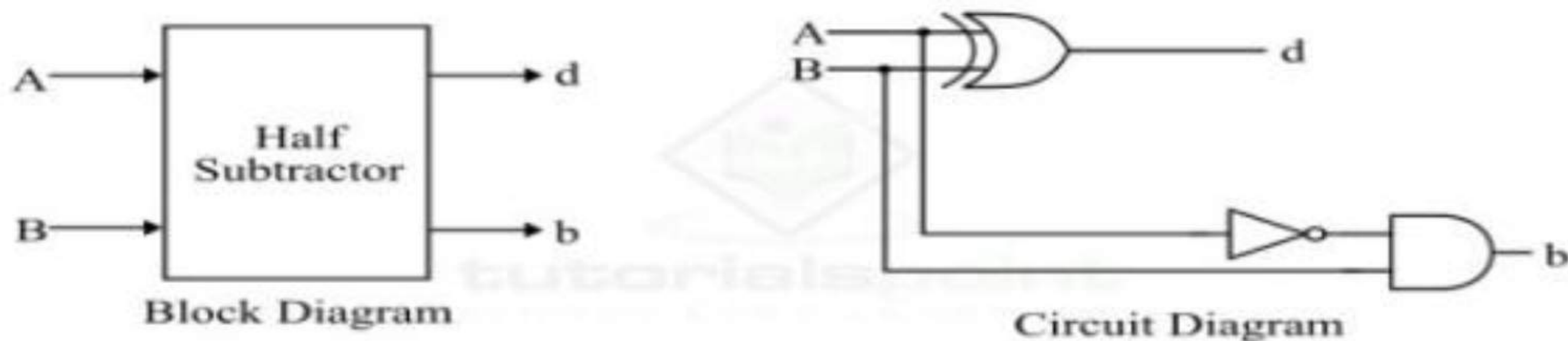


Figure 1 - Half Subtractor

Hence, from the logic circuit diagram, it is clear that a half subtractor can be realized using an XOR gate together with a NOT gate and an AND gate.

In the half subtractor as shown in figure-1, A and B are the inputs, d and b are the outputs. Where, d indicates the difference and b indicates the borrow output. The borrow output (b) is the signal that tells the next stage that a 1 has been borrowed.



Operation of Half Subtractor

Now, let us understand the operation of the half subtractor circuit. Half subtractor performs its operation to find the difference of two binary digits according to the rules of binary subtraction, which are as follows –

The output borrow of b is zero (0) as long as the minuend bit (A) is greater than or equal to the subtrahend bit (B), i.e. $A \geq B$. The output borrow is a 1 when $A = 0$ and $B = 1$.

From the logic circuit diagram of the half subtractor, it is clear that the difference bit (d) is obtained by the XOR operation of the two inputs A and B , and the borrow bit is obtained by AND operation of the complement of the minuend (A') with the subtrahend (B).



Truth Table of Half Subtractor

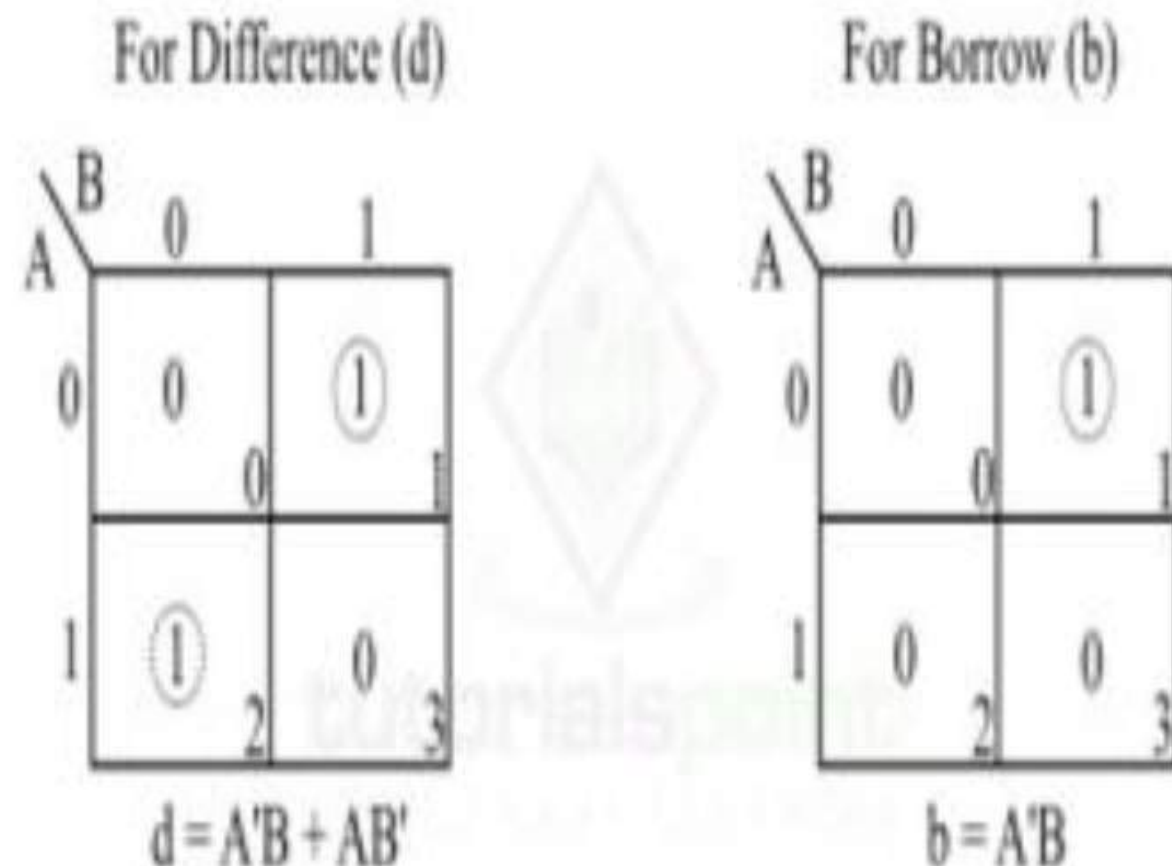
The following is the truth table the half-subtractor –

Inputs		Outputs	
A	B	D (Difference)	B (Borrow)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

K-Map for Half Subtractor

We can use the K-Map (or Karnaugh Map), a method for simplifying Boolean algebra, to determine equations of the difference bit (d) and the output borrow (b).

The K-Map simplification for half subtractor is shown in Figure-2.





Characteristic Equation of Half Subtractor

The characteristic equations of the half subtractor, i.e. equations of the difference bit (d) and the output borrow bit (b) are obtained by following the rules of binary subtraction. These equations are given as follows –

The difference bit (d) of the half subtractor is given by XORing the two inputs A and B. Therefore,

$$\text{Difference, } d = A \oplus B = A'B + AB'$$

The borrow (b) of the half subtractor is the AND of A (compliment of A) and B. Therefore,

$$\text{Borrow, } b = A'B$$



Applications of Half Subtractor

The following are some important applications of half subtractor –

- Half subtractor is used in ALU (Arithmetic Logic Unit) of processors.
- Half subtractor can also be used in amplifiers to compensate the sound distortion.
- It is also used to decrease the force of radio signals or audio signals.
- Half subtractor is also used to increase or decrease operators.

Conclusion

From the above discussion, we can conclude that a half subtractor is a combinational logic circuit that can calculate the difference of two binary digits. A half subtractor can only be used to subtract the LSB (Least Significant Bit) of the subtrahend from the LSB of the minuend when one binary number is subtracted from another binary number.

THANK YOU